MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

by

Group 1

Dương Bình An	Lê Quang Tuyến
Phạm Đức Long	Đỗ Quốc Trung
Dương Hồng Quân	Nguyễn Lê Hồng Nhi

A thesis submitted in conformity with the requirements

for the degree of Master of Software Engineering

MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

by

Group 1

Dương Bình An	Lê Quang Tuyến
Phạm Đức Long	Đỗ Quốc Trung
Dương Hồng Quân	Nguyễn Lê Hồng Nhi

A thesis submitted in conformity with the requirements

for the degree of Master of Software Engineering

Supervisor:

1. TAN Le Duy
2.

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

Group 1

Dương Bình An
Lê Quang Tuyến
Phạm Đức Long
Đỗ Quốc Trung
Dương Hồng Quân
Nguyễn Lê Hồng Nhi

Degree Master of Software Engineering

FPT University

2026

Abstract

This thesis presents an academic fraud risk scoring workflow for the IEEE-CIS dataset. Apache Spark performs typed ingestion, transaction–identity joining, quality auditing, chronological leakage-controlled processing, feature engineering, and Parquet handover. Python model training compares Logistic Regression, LightGBM, XGBoost, and CatBoost after the Spark-to-pandas boundary. The system packages feature mappings, calibration, thresholds, and explanations for an in-process FastAPI service with a React review interface.

The current CatBoost balanced candidate reports validation ROC-AUC 0.9081210 and PR-AUC 0.5805993, with holdout ROC-AUC 0.8780755 and PR-AUC 0.4266435. The current CatBoost candidate is labelled `v2` in its metadata, but its promotion decision is `not_promoted`; the configured V2 software default remains unchanged. The principal limitations are offline–online historical-feature mismatch, duplicated policy ownership, local lineage, single-node training, fail-open fallback, and incomplete security and monitoring controls. The result is a bounded academic demonstration, not a production-readiness claim.

ACKNOWLEDGMENTS

The members of Group 1 sincerely thank their supervisor, TAN Le Duy, for the guidance and feedback provided during the development and documentation of this project.

The project team consists of Dương Bình An, Lê Quang Tuyến, Phạm Đức Long, Đỗ Quốc Trung, Dương Hồng Quân, and Nguyễn Lê Hồng Nhi. The repository records subsystem responsibilities for data processing and exploratory analysis, model development, backend engineering, and frontend engineering. The interfaces and handover contracts between these areas made it possible to evaluate the system as one end-to-end artifact.

CONTENTS

CHAPTER 1

INTRODUCTION

1.1 Problem context

Online transaction fraud is a highly imbalanced classification problem in which the positive class is rare, errors have asymmetric consequences, and the data distribution can change over time. A useful operational system must therefore do more than maximize a single offline metric. It must preserve the temporal meaning of validation, make feature transformations reproducible, expose model version and uncertainty, support investigation, and connect predictions to a review process.

The IEEE-CIS Fraud Detection dataset was released through a Kaggle competition and contains anonymized card-not-present transaction and identity information provided by Vesta Corporation (IEEE Computational Intelligence Society, 2019; Amazon Science, 2023). Its scale and mixture of numeric, categorical, missing, and identity attributes make it a useful academic basis for studying big-data preparation and tabular machine learning.

This project addresses the following system question:

> How can a reproducible big-data pipeline, an imbalanced-classification model, an explainable scoring API, and a human-review interface be integrated into one auditable fraud risk scoring demonstration?

1.2 Objectives

The project has six technical objectives:

1. ingest and validate the full IEEE-CIS transaction and identity files;
2. build leakage-controlled, chronological, model-ready datasets;
3. compare interpretable and gradient-boosted classifiers using metrics suitable for an imbalanced positive class;
4. package the selected estimator behind a stable per-transaction scoring interface with local explanations;
5. expose scoring, persistence, review, import, and dataset loading through a typed API;
6. provide a usable dashboard and reproducible local deployment.

1.3 Research and engineering questions

The implementation and review are organized around four questions:

RQ1: Can the raw data be transformed into a versioned handoff without identifier overlap or training-to-holdout leakage?

RQ2: Which evaluated classifier has the strongest saved validation PR-AUC, and does its final artifact retain performance on the chronological holdout?

RQ3: Can the batch model be served without Spark at request time while retaining categorical compatibility and local explanations?

RQ4: Which gaps prevent the demonstration from supporting a production-readiness claim?

1.4 Contributions

The project contributes:

- a Spark pipeline with typed ingestion, join audits, EDA, temporal splitting, training-only transform fitting, feature contracts, and a machine-readable verifier;
- weighted and deterministic undersampling strategies that leave validation and holdout prevalence untouched;
- a versioned comparison of logistic regression, LightGBM, XGBoost, and CatBoost, with a packaged version-aware scoring interface;
- a FastAPI and SQL persistence layer for scoring, imports, dataset loads, transaction exploration, and human review;
- a React dashboard with risk visualization, SHAP evidence, data loading, review operations, and fallback-aware states;
- a whole-project audit that separates verified behavior, saved artifact claims, implemented capability, and missing evidence.

1.5 Evaluation basis

The primary predictive metrics are ROC-AUC and precision–recall AUC (PR-AUC). PR curves are particularly informative when positive and negative class counts are highly skewed because precision exposes the false-positive burden among positive predictions (Davis and Goadrich, 2006; Saito and Rehmsmeier, 2015).

The project does not treat area metrics as a complete operational policy. Precision, recall, confusion counts, calibration, error cost, and review capacity at a selected threshold are required before a model score can control real financial decisions.

1.6 Scope and evidence

The evidence snapshot is the repository and locally available generated artifacts inspected on 31 July 2026. Source code proves that a capability is implemented; it does not by itself prove that a training or deployment run completed. Saved JSON/CSV artifacts are reported as when they could not be independently regenerated in the review environment.

The thesis excludes:

- claims about real production fraud savings;
- personal or merchant identification from anonymized fields;
- causal explanations of fraud;
- automatic deployment or online learning;
- regulatory approval of the risk-band policy.

1.7 Thesis organization

Chapter 2 introduces the dataset, large-scale processing, boosted-tree models, evaluation, and SHAP. Chapter 3 defines requirements and the integrated architecture. Chapter 4 examines the data pipeline and its leakage controls. Chapter 5 presents model training, results, serving, and limitations. Chapter 6 documents the API, database, frontend, and deployment. Chapter 7 consolidates validation evidence and the production-readiness review. Chapter 8 answers the research questions and identifies future work.

CHAPTER 2

BACKGROUND AND RELATED WORK

2.1 Fraud detection as imbalanced, temporal ranking

The project treats fraud detection as a binary ranking problem with a rare positive class. Accuracy is therefore insufficient: a majority-class predictor can appear strong while detecting no fraud. PR-AUC is used as the primary model selection metric, with ROC-AUC as a complementary ranking measure. Precision and recall remain relevant because the backend exposes review and reject decisions. These metrics describe predictive behaviour; they do not measure fraud loss, investigation cost, or customer impact.

The second constraint is time. Labels and entity histories are associated with transactions ordered by `TransactionDT`. Random splitting could allow future distributions or histories to influence development evidence. The pipeline therefore uses chronological windows and train-only fitting. This is the methodological background needed to interpret the later leakage-control and holdout sections.

2.2 IEEE-CIS data characteristics

IEEE-CIS separates transaction data from optional identity data and includes categorical, numeric, missingness, and anonymized behavioural fields. The inspected training transaction file contains 590,540 rows and the observed fraud prevalence is approximately 3.5%. Because identity records are absent for many transactions, the project preserves the transaction table as the row authority and applies a left join. Identity presence and missingness are retained as features, but their association with fraud is not interpreted as causal evidence.

2.3 Technology boundary

Apache Spark is used for typed CSV ingestion, joins, quality profiling, chronological splitting, feature computation, and Parquet handover. The model matrix is then collected to pandas for Python estimators. The correct architecture statement is therefore: **distributed data preparation with single-node Python model training**. FastAPI and React provide the integrated scoring demonstration; they do not constitute independent model microservices.

The compared model families are Logistic Regression, LightGBM, XGBoost, and CatBoost. Their role in this report is methodological comparison rather than a claim of

distributed estimator training. Tree-based SHAP values are used as local model attributions. They describe the model output and are not causal explanations; when calibration changes the displayed probability, raw-model SHAP should not be interpreted as an explanation of the calibrated scale.

2.4 Scope boundary

The thesis evaluates an academic batch-to-serving workflow. Online historical feature generation, immutable registry authority, production monitoring, authentication, and operational policy approval are treated as limitations or target architecture, not as implemented capabilities.

CHAPTER 3

REQUIREMENTS AND SYSTEM ARCHITECTURE

3.1 Functional requirements

The implemented system satisfies the following demonstration requirements:

Table 3.1: *Functional requirements and implementation status*

ID	Requirement	Status
FR-01	Validate and process the required transaction and identity CSV files.	Implemented
FR-02	Produce train, validation, holdout, and unlabeled model-ready datasets.	Verified
FR-03	Preserve a canonical feature order and preprocessing metadata.	Verified
FR-04	Compare multiple binary classifiers using validation metrics.	Artifact-reported
FR-05	Score one transaction and return probability, risk, version, and explanation.	Implemented
FR-06	Persist scored transactions and human review outcomes.	Implemented
FR-07	Filter, sort, paginate, and inspect transactions.	Tested in API suite
FR-08	Import CSV and load processed datasets with progress reporting.	Implemented
FR-09	Support review approval/rejection and a fraud/legitimate label.	Tested in API suite
FR-10	Present risk, SHAP evidence, model state, and fallback state in a browser.	Implemented

3.2 Quality requirements

Table 3.2: *Quality requirements and assessment*

ID	Requirement	Assessment
QR-01	Temporal leakage control and non-overlapping splits.	Verified
QR-02	Reproducible schema and feature contracts.	Verified
QR-03	Version-aware model artifacts and rollback.	Implemented
QR-04	Explainable tree-model predictions.	Implemented
QR-05	Local deployment with bounded data mounts.	Implemented
QR-06	Automated subsystem tests.	Partial
QR-07	Auditable experiment and promotion lineage.	Incomplete
QR-08	Durable background operations.	Incomplete
QR-09	Authentication, authorization, and audit controls.	Not implemented
QR-10	Production monitoring and service objectives.	Not implemented

3.3 Integrated architecture

Figure 3.1 shows the implemented batch and serving layers. The central contract is the verified model-ready dataset plus its feature and schema metadata.

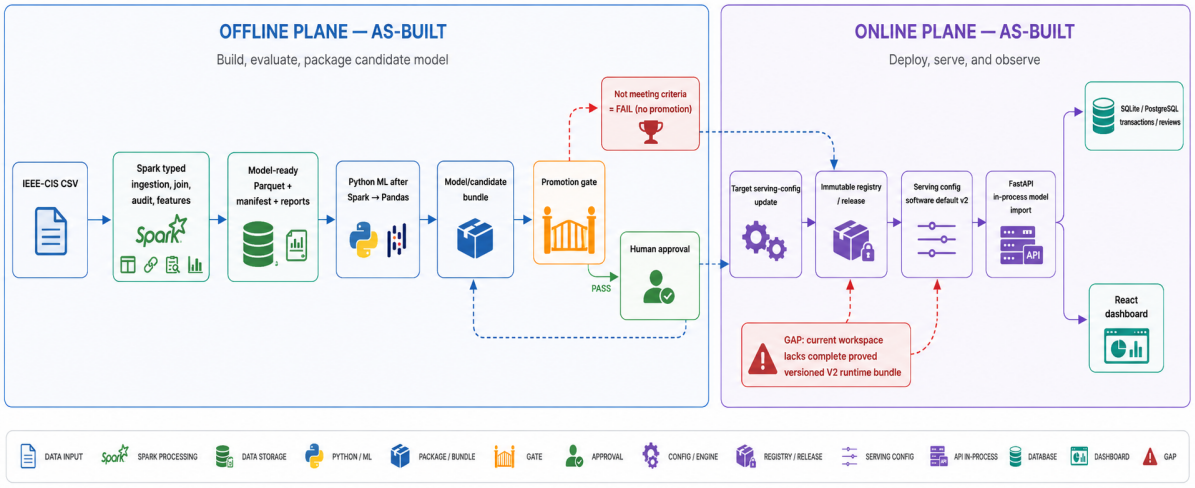


Figure 3.1: *Overall system architecture. Solid lines are implemented; dashed lines are proposed or partially enforced. The failed-candidate path leaves serving unchanged.*

The architecture uses a direct Python dependency between model and backend. This avoids network serialization and a separate model service for the demonstration. It also means backend dependency resolution and image size are coupled to the training project.

3.4 Component responsibilities

3.4.1 Data pipeline

The data component owns raw-file validation, typed ingestion, transaction–identity join preservation, quality and EDA reports, feature engineering, split policy, training-fitted transformations, Parquet output, and the manifest/verifier.

3.4.2 Model package

The model component owns dataset loading, training-only categorical indexing, candidate comparison, tuning, evaluation, artifact packaging, version path selection, SHAP initialization, and the `score(features)` interface.

3.4.3 Backend

The backend owns API validation, risk bands, automatic decisions, model/fallback state, SQL persistence, transaction queries, reviews, CSV import, processed dataset discovery, and background loading jobs.

3.4.4 Frontend

The frontend owns user navigation, filtering and pagination, risk presentation, SHAP visualization and accessible fallback, review input, ad-hoc scoring, data loading/import, polling, theme preferences, and rule-based assistance.

3.5 Primary data flows

3.5.1 Batch preparation

Raw CSV files are mounted into a Linux Spark container. The pipeline writes ignored local artifacts under the processed data directory. Downstream modules consume Parquet and JSON contracts, not in-memory Spark objects.

3.5.2 Training

The model loads the weighted training split and untouched validation/holdout splits, fits categorical mappings on training, converts the model matrix to pandas, selects a candidate on validation, and writes a versioned artifact.

3.5.3 Real-time scoring

The backend calls the in-process model package. Raw string categorical values are mapped using dictionaries persisted with the artifact. Probability and local SHAP evidence are mapped into risk presentation and returned through Pydantic schemas.

3.5.4 Review

Scored transactions are stored before investigation. The human review result is a separate entity so an automatic decision remains distinguishable from a reviewer's status and label.

3.6 Contracts

The system has three principal contracts:

1. the model-ready data contract, including canonical feature order and schema/version metadata;
2. the model scoring contract, including required/defaultable features, probability, SHAP, and version metadata;
3. the HTTP contract, including transaction, review, dataset, load, job, import, and scoring types.

The first contract is strongly machine-checked. The third is manually duplicated between Pydantic, TypeScript, mocks, and Markdown. At the evidence snapshot, the TypeScript declarations omit several backend metadata fields and `scoring_mode`; OpenAPI-generated types are recommended.

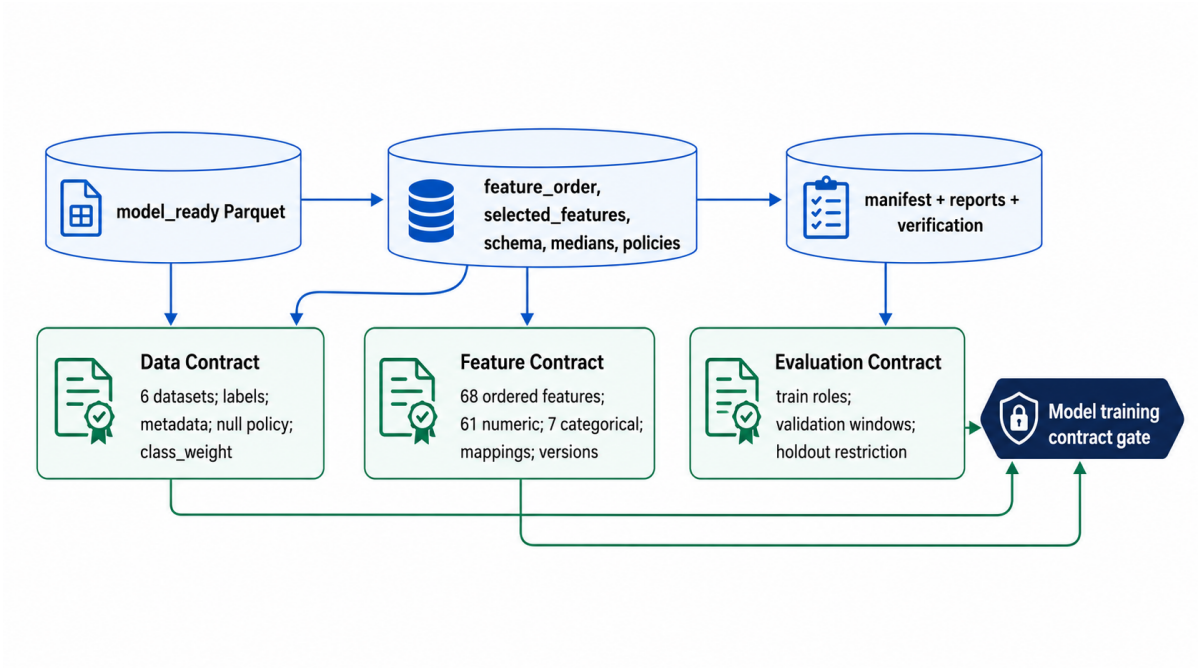


Figure 3.2: *Data, feature, and evaluation contracts. Split configuration and training protocol define the Evaluation Contract.*

3.7 Risk and trust boundaries

The browser is untrusted input. FastAPI/Pydantic validates JSON and route parameters; CSV parsing applies bounded row limits and per-row error handling. The database and processed-data mount are trusted local resources.

Joblib model artifacts are also a trust boundary because Python deserialization can execute code. Only artifacts produced by a controlled training pipeline should be loaded. The current CatBoost candidate stores SHA-256 checksums, but the repository does not yet provide an external immutable registry that is the runtime serving authority.

The fallback heuristic is a further trust concern. It keeps the demonstration available when the model fails, but it must not be allowed to impersonate a production model. The backend exposes a warning, yet readiness remains fail-open. This is a degraded demo mode, not a production-equivalent score.

3.8 Architectural tradeoffs

Table 3.3: *Major architectural tradeoffs*

Choice	Benefit	Cost
Monorepo and path dependency	Simple handoff and one Compose stack	Backend inherits model dependencies
Spark then pandas	Scalable preparation and familiar estimators	Driver-memory training ceiling
Fixed backend risk bands	Stable UI and simple tests	Not tied to model operating evidence
In-memory jobs	Minimal infrastructure	Lost on restart and single-worker only
SQLite local default	Fast developer startup	Different operational behavior from PostgreSQL
Heuristic fallback	Demo continuity	Can conceal model loading failures
Manual TypeScript types	No generator setup	Contract drift risk

CHAPTER 4

BIG-DATA PROCESSING AND FEATURE ENGINEERING

4.1 Source validation and typed ingestion

The raw inventory contains the four required IEEE-CIS files and an optional sample-submission file. The four required files occupy approximately 1.29 GB. The pipeline validates discovery, required names, sizes, headers, and schemas before performing expensive Spark actions.

Table 4.1: *Observed source files*

File	Bytes	Role
`train_transaction.csv`	683,351,067	Labeled transactions
`train_identity.csv`	26,529,680	Optional train identity
`test_transaction.csv`	613,194,934	Unlabeled transactions
`test_identity.csv`	25,797,161	Optional test identity

Explicit Spark schemas avoid inferring a different type across runs and make conversion failures measurable. Identity column names are normalized before the join.

4.2 Join integrity

The transaction table is the authoritative row set and identity is joined with a left join on `TransactionID`. Table 4.2 shows that the join preserves every transaction row.

Table 4.2: *Transaction–identity join audit*

Dataset	Transactions	Identity matches	Unmatched	Row difference
Train	590,540	144,233	446,307	0
Test	506,691	141,907	364,784	0

The large unmatched count is expected. Presence and missingness features preserve information about identity availability without dropping rows.

4.3 Exploratory analysis and quality controls

The pipeline writes source inventory, key and join audits, duplicate summaries, invalid-record summaries, missingness profiles, class distributions, temporal aggregations, product/card/device/email breakdowns, and outlier reports.

These reports are descriptive. A higher observed fraud rate for a device or product group does not establish a causal relationship; the group can be confounded by transaction mix, time, or missingness.

4.4 Leakage-controlled transformation order

Figure 4.1 highlights the transforms whose parameters are fitted only on the training window.

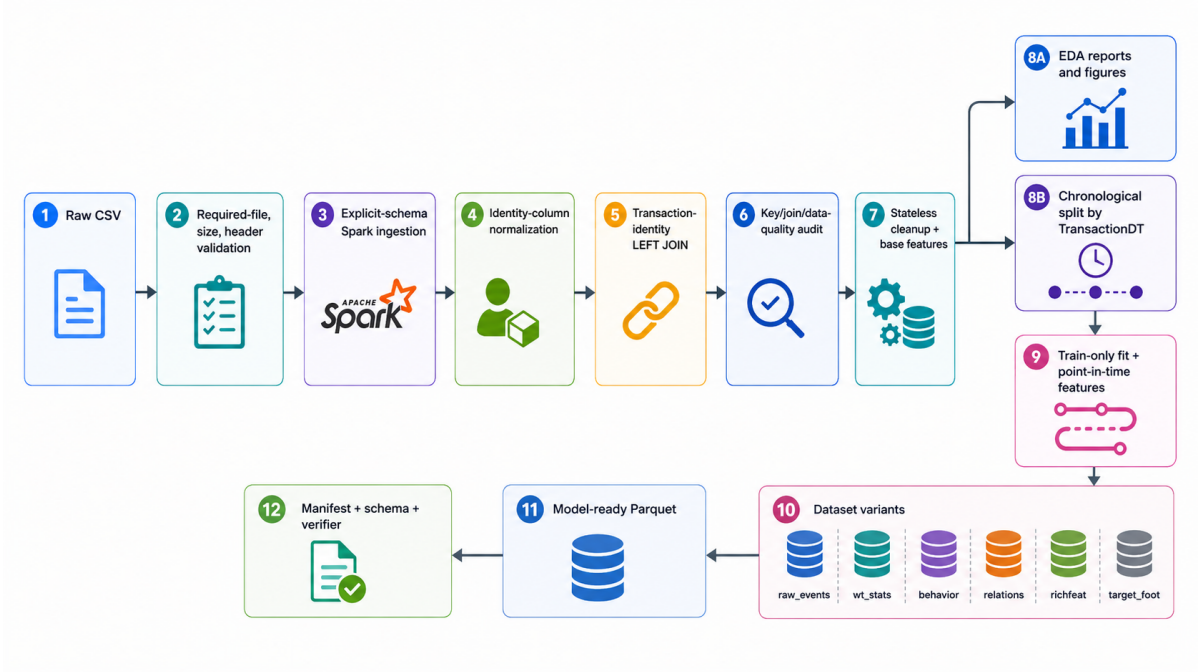


Figure 4.1: *Detailed data processing pipeline. Shows canonical dataset names, Spark transformation stages, and output handover contracts.*

The use of `TransactionDT` produces forward validation rather than a random split. This better represents the intended question: whether a model trained on earlier transactions ranks later transactions.

4.5 Chronological split

Table 4.3: *Chronological labeled splits*

Dataset	Rows	Fraud	Fraud rate	`TransactionDT` range
Train	412,956	14,522	3.5166%	86,400–10,432,902
Validation	88,486	3,036	3.4310%	10,432,915–13,136,583
Holdout	89,098	3,105	3.4849%	13,136,627–15,811,131

The saved verifier found unique identifiers inside each split and zero overlap between split pairs. The nearly stable prevalence across natural windows makes the comparison interpretable, although one validation window cannot quantify variation across multiple historical regimes.

4.6 Feature engineering

4.6.1 Time features

Time features include transaction day, week, hour, a day-of-week proxy, age in days, and a night-transaction flag. The raw `TransactionDT` is retained in the canonical feature order.

4.6.2 Amount features

Amount processing includes the raw value, logarithm, capped value, decimal component, amount band, and ratios/deviations relative to card history. Saved outlier thresholds are learned from training and applied forward.

4.6.3 Missingness and presence

The pipeline produces counts and ratios for selected/identity missing values and flags for identity, device, payer/recipient email, distance, address, and shared email domain presence.

4.6.4 Anonymized behavior fields

Selected `C1--C14`, `D1--D15`, and distance features are retained. Their public semantic meaning is limited, so this report describes them as behavioral/anonymized variables rather than inventing business definitions.

4.6.5 Historical entity features

Card, email, and device keys produce prior transaction counts and amount aggregates. Card history additionally includes standard deviation and time since the previous transaction. Training rows use prior-window history; later splits receive lookups fitted only from the training window.

4.6.6 Categorical features

The seven categorical fields are:

- `ProductCD`, `card4`, and `card6`;
- `DeviceType` and `device_family`;
- `M4` and `amount_band`.

String indexers are fitted on training. The extracted mappings are serialized with the model for Spark-free request-time encoding.

4.7 Missing-value and class-imbalance strategies

Numeric medians and categorical missing policies are persisted. At serving, missing values that remain after categorical mapping use a numeric sentinel. This is compatible with the current tree models, but the sentinel must remain part of the model contract.

The primary training set uses class weights and keeps all 412,956 training rows. A second set undersamples legitimate examples:

Table 4.4: *Training and evaluation class distributions*

Dataset	Legitimate	Fraud	Legitimate-to-fraud ratio
`train_original`	398,434	14,522	27.44
`train_weighted`	398,434	14,522	27.44
`train_balanced`	43,872	14,522	3.02
`validation`	85,450	3,036	28.15
`holdout`	85,993	3,105	27.70

Validation and holdout remain untouched. This is essential because precision and calibration depend on class prevalence.

4.8 Output contract

The pipeline exports curated data, splits, model-ready Parquet, feature-store tables, preprocessing artifacts, schema artifacts, reports, and demonstration cases. The

canonical model-ready datasets are:

- `train_original`;
- `train_weighted`;
- `train_balanced`;
- `validation`;
- `holdout`;
- `kaggle_test`.

4.9 Verification result

The saved verifier reports 94 checks passed and no failures. It validates dataset presence, row counts, identifiers, labels, weights, feature order, schema hashes, chronological order, split overlap, and required artifacts.

The current official-run report records the terminal verifier result:

```
ready_for_downstream_training
```

An older persisted manifest/snapshot used `verification_pending`; it is treated as historical evidence and must not be mixed with the official-run report when generating final metrics.

4.10 Data limitations

- Feature semantics are partly anonymized.
- Entity aggregates are batch features; a production online path is not implemented.
- The training matrix is collected to pandas before estimator fitting.
- There is one chronological validation window rather than temporal cross-validation.
- Native Windows is not the canonical full Parquet execution path.
- The local processed tree is large and intentionally excluded from Git.

CHAPTER 5

MODEL DEVELOPMENT, EVALUATION, AND SERVING

5.1 Artifact identity and terminology

This report separates four states that are otherwise easy to conflate. The historical V1/V2 comparison refers to earlier LightGBM artifacts and reports. The configured V2 software default is the version selected by serving configuration, not a promotion approval. The current official candidate is a CatBoost balanced package whose metadata also uses the version label `v2`; it is therefore called the *current CatBoost candidate* in this chapter. Its promotion decision is `not_promoted`, so this report does not claim that the candidate replaced the configured serving default.

Table 5.1: *Model identifiers and runtime roles*

Identifier	Model	Role	Status
Historical V2	LightGBM	Historical artifact	Historical
Serving default `v2`	LightGBM	Configured champion	Unchanged
Current candidate	CatBoost balanced	Challenger	Not promoted

5.2 Model-development protocol

The enhanced candidate workflow consumes the Spark/Parquet handover and executes the following ordered stages:

1. validate the data contract and canonical feature order;
2. fit categorical mappings on training data only;
3. train a Logistic Regression baseline;
4. compare Logistic Regression, LightGBM, XGBoost, and CatBoost, including balanced variants;
5. select and tune a candidate using the selection window;
6. fit an isotonic calibrator on the calibration window;
7. choose review and reject thresholds on the policy window;
8. freeze estimator, mappings, calibrator, and policy metadata;
9. evaluate the frozen package on holdout;
10. write candidate metadata, checksums, smoke evidence, and a promotion decision.

The feature loader removes `TransactionID`, `isFraud`, `class_weight`, `split_name`, processing metadata, and `generated_at` from the model vector. The canonical vector contains 68 features, 61 numeric and 7 categorical, and includes `TransactionDT`. This prevents a previous contract failure in which the string `train_weighted` entered the numeric training frame.

The boundary must be stated precisely as: **distributed data preparation with single-node Python model training**. Spark writes Parquet; Python training converts model matrices to pandas. There is no evidence that LightGBM, XGBoost, or CatBoost is distributed over Spark workers.

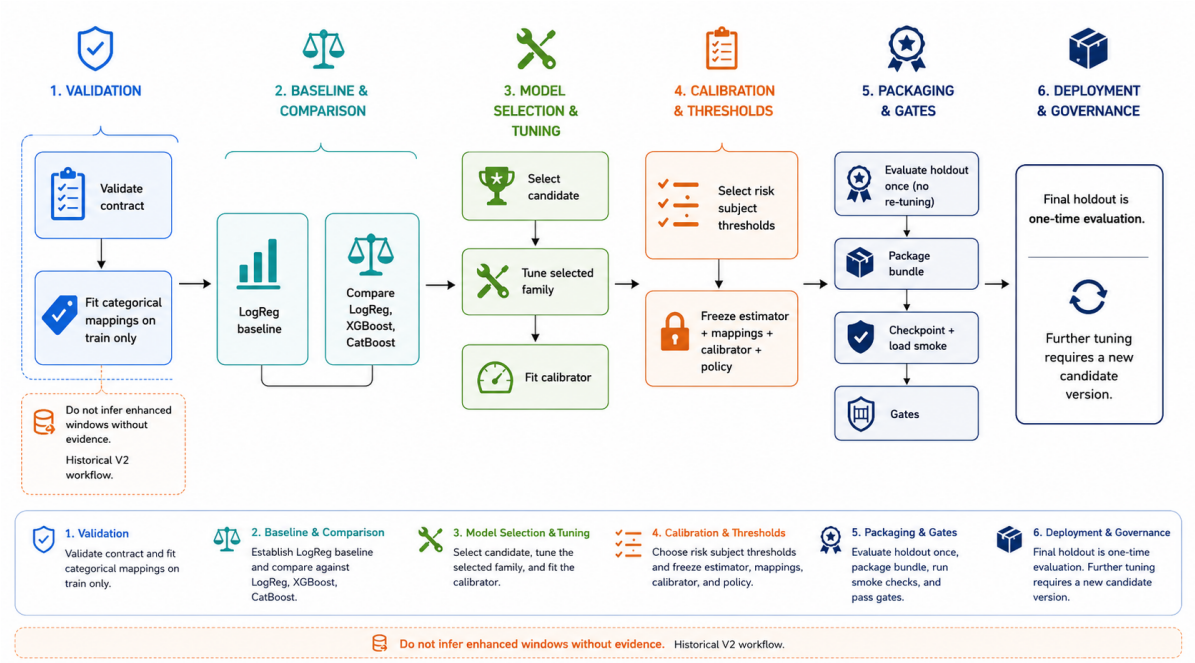


Figure 5.1: *Model training lifecycle. Spark provides distributed data preparation; estimator fitting is single-node Python training.*

5.3 Candidate models and selection

The baseline is Logistic Regression. The compared gradient-boosting families are LightGBM, XGBoost, and CatBoost. The current CatBoost candidate comparison selects `catboost__balanced`; the candidate manifest identifies the final model as CatBoost trained on the balanced variant.

Table 5.2: *Current CatBoost candidate training evidence*

Field	Value
Artifact version label	`v2`
Selected model	CatBoost
Training variant	`balanced`
Feature count	68
Selection window	44,243 rows
Calibration window	22,122 rows
Policy window	22,121 rows
Calibration method	Isotonic
Processing version	`ieee-cis-preprocess-2.1.0`
Feature schema	`ieee-cis-features-1.1.0`

The saved comparison artifact contains model-family and balanced-variant results. PR-AUC is the primary selection metric because the positive class is rare. ROC-AUC is retained as a complementary ranking metric. Accuracy is not used as a selection authority. Validation comparison is descriptive evidence; it does not establish a business operating point.

Table 5.3: *Current model-family selection evidence*

Model	Training variant	ROC-AUC	PR-AUC	Decision
Logistic Regression	Weighted	0.8219	0.3090	Baseline
LightGBM	Weighted	0.9116	0.5682	Compared
XGBoost	Weighted	0.8879	0.5529	Compared
CatBoost	Weighted	0.8757	0.5245	Compared
CatBoost	Balanced	0.9070	0.5703	Selected

CatBoost Balanced was selected because it achieved the highest selection-window PR-AUC, which was the primary metric for this imbalanced dataset.

5.4 Temporal development windows

The validation period is divided into selection, calibration, and policy windows. The current candidate metadata records selection boundary `11713696` and calibration boundary `12438149`. Model family and parameters are selected in the first window, isotonic calibration is fitted in the second, and thresholds are selected in the third. Holdout remains the final evaluation window and is opened after the package is frozen.

This separation is stronger than the historical V1 workflow, but it should not be used to rewrite V1's history. If a model is modified after holdout results are viewed, the

modified package must receive a new candidate identity rather than silently reusing the same holdout.

5.5 Current CatBoost candidate results

Table 5.4: *Current CatBoost candidate validation and holdout metrics*

Window	Threshold	PR-AUC	ROC-AUC	Precision	Recall	Bri
Validation selection	0.50	0.5805993	0.9081210	0.3969649	0.6220275	
Holdout	0.16	0.4266435	0.8780755	0.3820038	0.5181965	0.024832

Figures 7.3a–7.4 summarize the candidate's holdout evidence. All values are artifact-reported from `reports/official-run/validation_metrics_v2.csv`, `holdout_metrics_v2.csv`, and `training_metadata_v2.json`.

The current CatBoost candidate is not directly comparable to the historical LightGBM V1/V2 tables. The official candidate is CatBoost balanced, whereas older Markdown reports describe V2 as LightGBM and cite holdout PR-AUC 0.4582. The old values are therefore historical evidence, not metrics for the current CatBoost candidate. A matching V1 final holdout artifact is not present in the current repository, so this chapter does not invent a V1 holdout comparison.

5.6 Calibration and threshold policy

The current candidate stores `calibration_model_v2.joblib` and records `calibration_method=isotonic`. The threshold artifact selects:

Table 5.5: *Current candidate policy artifact*

Policy field	Value
Review threshold	0.16
Reject threshold	0.53
Minimum review precision	0.30
Minimum reject precision	0.60
Selected review precision	0.3371429
Selected reject precision	0.7397959

The model scorer applies the calibrator when the loaded artifact contains one. However, the backend runtime decision authority is the Risk Scoring module: it rounds probability to a 0–100 score, applies fixed bands, and maps those bands to approve, review, or reject. The backend does not load the candidate review/reject thresholds

as its decision policy. This is duplicated ownership, not evidence that the candidate policy is active in serving.

Table 5.6: *Candidate policy versus backend runtime policy*

Decision	Candidate policy	Backend runtime policy	Active authority
Review starts	0.16	0.40	Backend
Reject starts	0.53	0.80	Backend

Candidate thresholds are packaged as model evidence but are not currently the backend runtime decision authority. Holdout operating-point evidence is available for the review threshold of 0.16; reject-threshold holdout evidence was not available in the inspected artifacts.

5.7 Packaging, checksum, and promotion

The current candidate package includes the final joblib model, training metadata, calibrator, threshold configuration, validation and holdout metrics, threshold analysis, candidate manifest, promotion decision, and SHA-256 checksums. The candidate manifest records artifact-load smoke `passed`, model name `catboost`, feature count 68, processing/schema versions, MLflow run ID, Git commit, and `git_dirty=true`.

Table 5.7: *Current candidate promotion gate*

Gate	Result	Evidence
Holdout PR-AUC $\geq$ 0.4582	Fail	0.4266435
Holdout ROC-AUC $\geq$ 0.8746	Pass	0.8780755
Review precision $\geq$ 0.30	Pass	0.3820038
Calibration non-regression	Pass	Brier decreased
Artifact checksum	Pass	SHA-256 match
Artifact-load smoke	Pass	recorded in manifest
Data contract verification	Pass	ready, 94 checks
Version match	Pass	`v2`
Git clean	Fail	`git_dirty=true`

The resulting decision is `not_promoted`, with `serving_version_unchanged=true`. The current candidate therefore did not replace the configured V2 software default. This is a governance outcome, not a claim that the candidate became V2 or that the historical LightGBM V2 artifact is identical to the current CatBoost candidate.

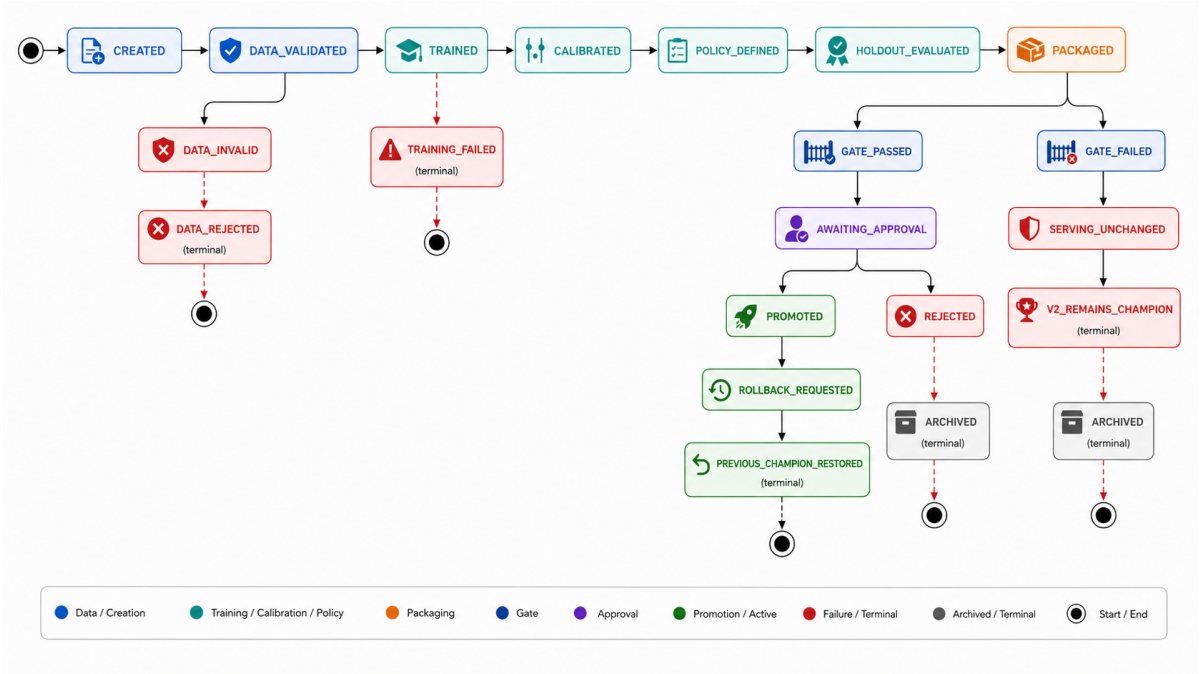


Figure 5.2: *Promotion state machine. A failed candidate enters a failure/archive path while serving configuration remains unchanged.*

5.8 Request-time scoring and explanations

The scorer loads one artifact into a process cache and maps raw categorical strings using training-derived dictionaries. It aligns columns to the canonical order, converts missing values to the configured sentinel, calls `predict_proba`, applies an available calibrator, and returns the top five absolute TreeSHAP contributions for supported tree models. SHAP is a local model attribution and not a causal explanation. When calibration changes the displayed probability, the current SHAP values still describe the raw model output unless a separate calibrated explanation method is used.

The response distinguishes `full_feature` from `partial_demo`. Full-feature scoring requires the canonical engineered vector. Partial-demo scoring defaults missing fields and is useful for UI continuity, but it cannot recreate card, email, and device histories from a small request. It must not be described as real-time full-feature scoring.

5.9 Serving configuration and MLflow

Configuration defaults `FRAUD_MODEL_SERVING_VERSION=v2`, while the current candidate decision remains `not_promoted`. The safe wording is “v2 configured software default”, not “production-promoted model”. Setting `FRAUD_MODEL_SERVING_VERSION=v1` remains the rollback/configuration path, subject to artifact compatibility.

MLflow is implemented as local experiment tracking and lineage using SQLite and local artifact storage. The repository does not evidence a central Model Registry, registered model versions, stage transitions, or an automatic promotion authority. Candidate

packaging and promotion JSON are evidence files, not a serving registry.

5.10 Model-readiness decision

The current CatBoost candidate is suitable for continued academic evaluation and manual review of its evidence. It should not be described as a production model until a frozen application smoke loads the expected artifact without fallback, the runtime policy is explicitly approved and aligned with the candidate, a durable registry/checksum handover exists, and online historical feature parity is addressed.

CHAPTER 6

APPLICATION AND DEPLOYMENT

6.1 Integrated serving architecture

The application is an **in-process model-serving modular monolith**. FastAPI imports the scoring package directly, persists scored transactions and review outcomes in PostgreSQL or SQLite, and exposes them to a React dashboard. This removes a network model-service boundary and is suitable for the demonstration. The tradeoff is coupling between API startup, model dependencies, image size, and scaling.

6.2 Scoring and review workflow

The backend validates requests, loads the configured scoring artifact, returns probability, risk score, fixed risk band, automatic decision, model metadata, and optional TreeSHAP values. Transactions retain the raw feature payload, score, timestamp, and model version. A separate review record stores reviewer status, fraud/legitimate label, and notes, so automatic decisions remain distinct from human outcomes.

The HTTP contract covers scoring, transaction exploration, CSV import, processed dataset loading, background jobs, and review. Endpoint-level paths and response fields are preserved in Appendix A and in the repository API contract. The main architecture issue is feature availability: full-feature scoring requires the canonical engineered vector, while partial-demo scoring defaults missing fields. The latter cannot recreate card, email, or device history from a small request and is not production-equivalent scoring.

6.3 Frontend integration

The React interface presents transaction lists, score details, risk bands, SHAP values, review actions, imports, and partial scoring. Mock responses can support UI development, but they are not model evidence. The interface uses text and numeric alternatives to colour and displays an explicit unavailable state when SHAP is absent. Figure 6.1 displays the operational fraud detection interface.

Figure 6.1: *Enterprise Fraud Detection and Risk Scoring Dashboard Interface.*

6.4 Deployment boundary

Docker Compose separates preprocessing, training, and application profiles. The backend receives processed Parquet through a read-only mount and installs the model package as a path dependency. Spark workers support distributed data preparation; estimator fitting remains single-node after the Parquet-to-pandas boundary. The deployment diagram is retained in Appendix B because it is implementation-level detail rather than a main architecture decision.

6.5 Operational limitations

The application currently has fail-open heuristic fallback, in-memory job state, synchronous CSV import, coupled backend/model dependencies, no authentication or audit event log, and no automated browser test suite. These limitations are consolidated with data and governance gaps in Chapter 7; they are not repeated as separate readiness claims throughout the thesis.

CHAPTER 7

VALIDATION AND WHOLE-SYSTEM REVIEW

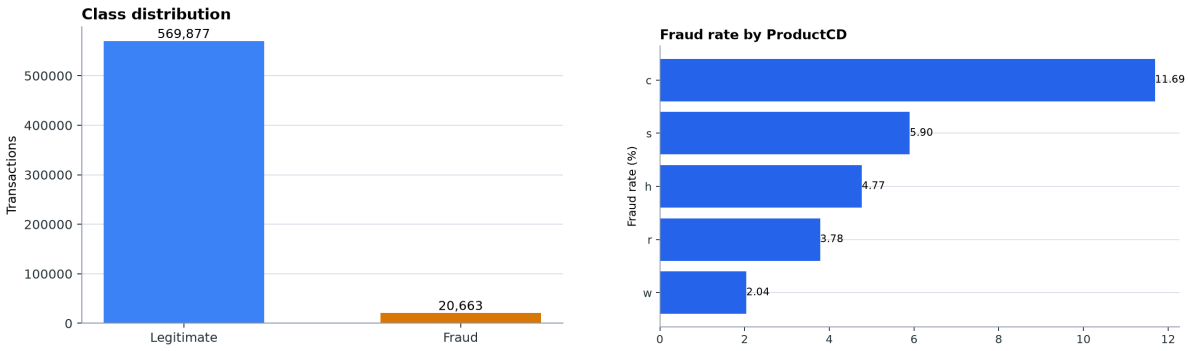
7.1 Evidence status

Claims in this thesis are classified as runtime-verified, artifact-reported, implemented from source, or proposed. The current official processed-data report records 94 verification checks passed and zero failed, with status `ready_for_downstream_training`. An older manifest/facts snapshot uses `verification_pending`; it is retained as historical evidence and is not combined with the official-run metrics.

7.2 Data and result evidence

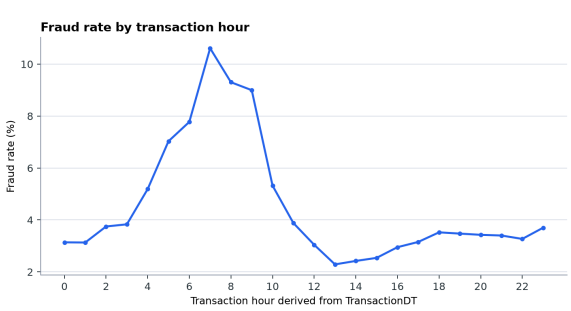
The verifier checks dataset presence, row counts, identifiers, labels, class weights, feature order, schema hashes, chronological order, split overlap, and required artifacts. The official candidate metadata reports 68 features, the selection/calibration/policy windows, isotonic calibration, holdout metrics, checksums, and a load-smoke result. These artifacts support the data-contract and candidate claims; they do not prove online feature parity or deployment readiness.

The following charts are descriptive evidence from the saved EDA and current candidate artifacts. They show class imbalance, selected group rates, model family selection, validation-to-holdout degradation, confusion counts, and calibration. None should be read as causal analysis.

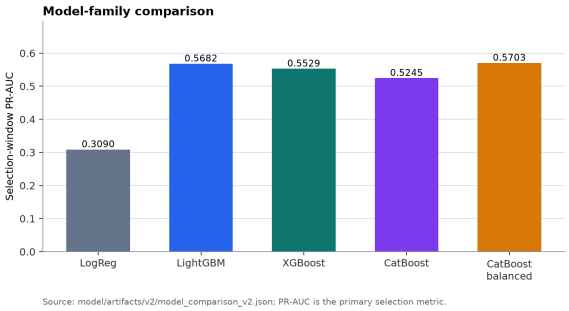


(a) *Class distribution.* **(b)** *Fraud rate by ProductCD.*

Figure 7.1: *Exploratory Data Analysis: Class distribution and fraud rate by ProductCD.*

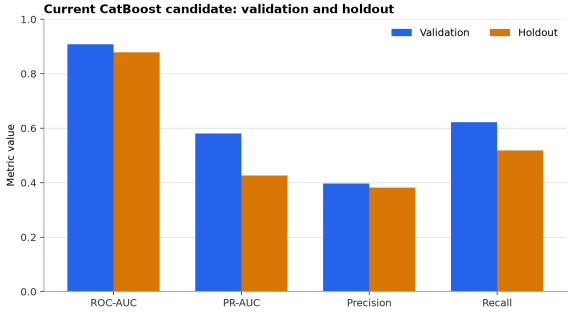


(a) *Fraud rate by transaction hour.*

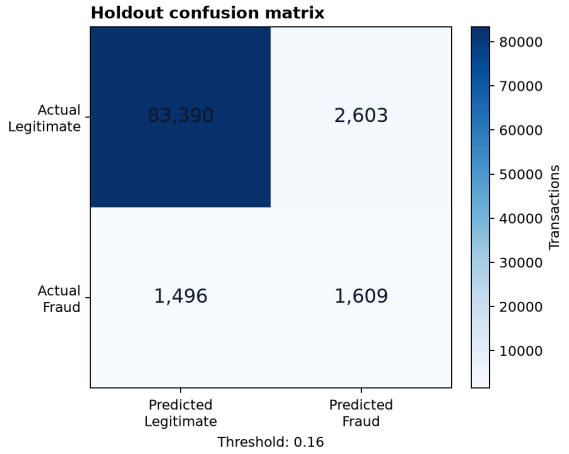


(b) *Model-family selection PR-AUC.*

Figure 7.2: *Temporal fraud association and model-family performance selection.*



(a) *Validation vs holdout metrics.*



(b) *Holdout confusion matrix (0.16 threshold).*

Figure 7.3: *Candidate holdout evaluation: Metrics drop and operating confusion matrix.*

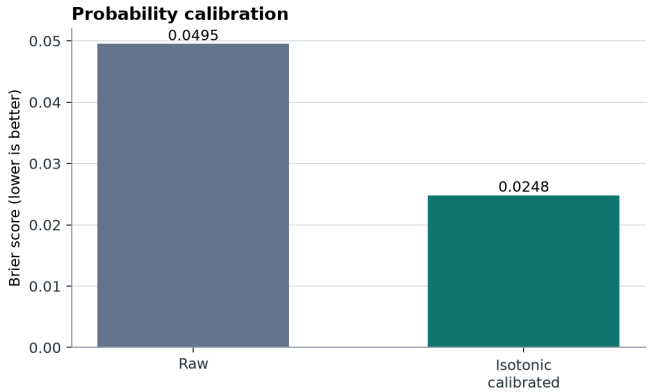


Figure 7.4: *Probability calibration: Raw vs isotonic-calibrated Brier score on holdout.*

7.3 Integrated validation findings

The strongest verified boundary is the Spark-to-Parquet handover: schemas, split membership, and learned preprocessing artifacts are available for downstream training. The strongest model evidence is the current CatBoost balanced candidate's

artifact-reported validation and holdout metrics. The strongest governance evidence is the explicit `not_promoted` decision and `serving_version_unchanged=true`; a failed candidate did not alter the configured V2 software default.

The application evidence is narrower. Source and subsystem tests cover model feature alignment, unseen categories, missing values, backend risk bands, and selected API behaviours. The stored full-stack execution history includes dependency and build remediation, but it is not a complete production smoke. Frontend lint/build evidence and model tests should therefore be described as subsystem evidence, not as proof of an end-to-end operational service.

7.4 Consolidated limitations

The review consolidates limitations into five decision-relevant groups:

1. **Reproducibility:** local MLflow tracking, workspace-local artifacts, snapshot discrepancies, and single-node pandas training limit portability and scale.
2. **Policy alignment:** the candidate threshold artifact and calibration metadata are not the backend's fixed risk-band authority.
3. **Offline–online parity:** batch historical aggregates are not generated by an online feature service; partial-demo defaults are not equivalent to full-feature scoring.
4. **Operational controls:** fallback is fail-open for demonstration continuity, jobs are in memory, and monitoring, migrations, and bulk re-scoring are absent.
5. **Security and governance:** authentication, authorization, durable audit, immutable registry authority, and production drift/latency monitoring are not evidenced.

These gaps define the boundary of the contribution. They do not invalidate the academic data and model workflow, but they prevent a production-readiness claim.

7.5 Recommended validation sequence

Before any future serving decision, reproduce the official candidate from a clean dependency and data snapshot, load the exact checksum without fallback, align the runtime policy with the approved calibration/threshold evidence, and test full-feature scoring with historical features generated by the same contract. Only then should an independent candidate version be compared and a human approval recorded.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Conclusion

The thesis demonstrates a coherent academic workflow from IEEE-CIS source files to Spark-processed Parquet, a versioned feature contract, Python model training, artifact-based evaluation, and an integrated FastAPI/React scoring demonstration. The main technical contribution is the separation of distributed data preparation from single-node estimator fitting, together with chronological leakage controls and a machine-checked handover.

The current CatBoost balanced candidate reports validation ROC-AUC 0.9081210 and PR-AUC 0.5805993, with holdout ROC-AUC 0.8780755 and PR-AUC 0.4266435. These values are artifact-reported. Its isotonic calibrator, policy thresholds, checksums, and promotion evidence are packaged, but the decision is `not_promoted`. The configured V2 software default therefore remains unchanged. This wording deliberately distinguishes the current CatBoost candidate from historical LightGBM V1/V2 reports and from serving approval.

8.2 Answers to the engineering questions

The data plane is technically coherent for the BDA501 objective: typed Spark ingestion, transaction-grain-preserving left joins, chronological splits, train-only transformations, point-in-time histories, and Parquet contracts are implemented or evidenced by saved artifacts. The model plane provides baseline and family comparison, selection/calibration/policy windows, holdout evaluation, candidate packaging, and promotion-gate evidence. The serving plane proves integrated scoring and review interaction, but not online reconstruction of historical features.

The main limitations are offline–online feature mismatch, duplicated policy ownership, local-only lineage, single-node training, fail-open fallback, and missing security and monitoring controls. These are architecture boundaries, not claims that another estimator would automatically resolve them.

8.3 Prioritized future work

Future work should proceed in this order:

1. freeze a reproducible candidate, checksum, image, data snapshot, and serving

smoke without fallback;
2. separate the approved model bundle from a versioned policy bundle and make runtime policy ownership explicit;
3. implement an online historical-feature path or restrict the API to demonstrably complete feature vectors;
4. add durable registry, audit, readiness, monitoring, security, and job lifecycle controls;
5. evaluate later candidates with temporal cross-validation and new candidate identities whenever holdout evidence has already been viewed.

The appropriate final assessment is therefore: the core Spark data and model development architecture is suitable for the academic objective, while the serving and governance layers remain a bounded demonstration with explicit limitations. No production-readiness claim is made.

BIBLIOGRAPHY

Amazon Science. Fraud dataset benchmark: Ieee-cis fraud detection, 2023. URL https://github.com/amazon-science/fraud-dataset-benchmark.

Jesse Davis and Mark Goadrich. The relationship between precision-recall and roc curves. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 233–240, 2006. doi: 10.1145/1143844.1143874.

IEEE Computational Intelligence Society. Ieee-cis fraud detection, 2019. URL https://www.kaggle.com/competitions/ieee-fraud-detection/overview.

Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015. doi: 10.1371/journal.pone.0118432.

CHAPTER A

API REFERENCE

A.1 Common enums

```
RiskBand = low | guarded | medium | high | critical
Decision = approve | review | reject
ReviewState = pending | approved | rejected
ReviewLabel = fraud | legit
ScoringMode = full_feature | partial_demo
```

A.2 Core response fields

Table A.1: *Core API fields*

Field	Type	Meaning
`fraud_probability`	float	Model/fallback output in $[0, 1]$
`risk_score`	integer	Rounded probability times 100
`risk_band`	enum	Fixed backend score band
`decision`	enum	Automatic backend policy result
`scoring_mode`	enum	Full vector or partial demonstration
`shap_top5`	list/null	Local model contributions when available
`model_version`	string	Artifact version used for the score
`processing_version`	string/null	Data processing lineage
`feature_schema_version`	string/null	Feature contract lineage

A.3 Endpoint summary

Method	Path	Response
GET	`/health`	Process status
GET	`/meta`	Model and feature metadata
POST	`/score`	Score response
GET	`/transactions`	Paginated transactions
GET	`/transactions/stats`	Aggregate statistics
GET	`/transactions/import/template`	UTF-8 CSV

Method	Path	Response
POST	`/transactions/import`	Import outcome and coverage
GET	`/transactions/{id}`	Transaction detail
POST	`/transactions/{id}/review`	Updated detail
GET	`/datasets`	Available datasets
POST	`/data/load`	Background job
GET	`/jobs/{id}`	Job state
GET	`/jobs/active/current`	Job or null
POST	`/jobs/{id}/cancel`	Updated job

The normative, implementation-aligned contract is maintained in the repository API contract specification.

CHAPTER B

DEPLOYMENT AND IMPLEMENTATION DIAGRAMS

B.1 Application deployment

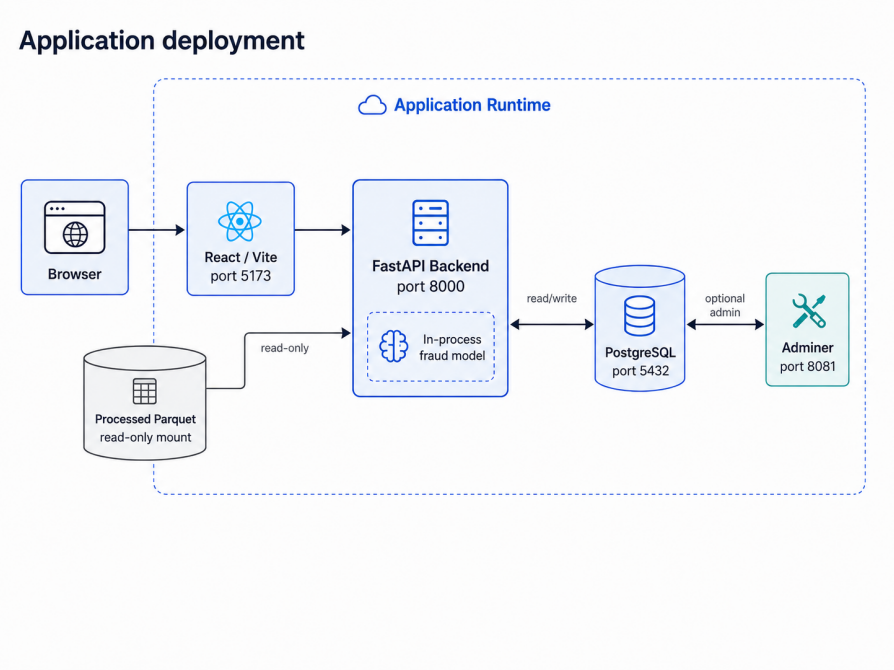


Figure B.1: *Default application deployment. Browser, React/Vite, FastAPI backend, PostgreSQL, and in-process fraud model.*

The application diagram is implementation-level evidence for the modular monolith. Spark preprocessing and Python model training are separate profiles; the diagram does not imply distributed estimator fitting.

CHAPTER C

FEATURE AND RISK CONTRACTS

C.1 Canonical feature groups

Table C.1: *Canonical feature groups*

Group	Representative features
Amount	`TransactionAmt`, log/capped/decimal amount, amount band, card-mean ratios and deviations
Time	`TransactionDT`, day, week, hour, day-of-week proxy, age, night flag
Missingness	Selected/identity missing counts and ratios
Presence	Identity, device, payer/recipient email, distance, address, same-domain flags
Anonymous behavior	`dist1`, `dist2`, selected `C1--C14`, selected `D1--D15`
Card history	Prior count/sum/mean/stddev and time since previous transaction
Email history	Prior count/sum/mean
Device history	Prior count/sum/mean
Categorical	`ProductCD`, `card4`, `card6`, `DeviceType`, `device_family`, `M4`, `amount_band`

C.2 Forbidden model columns

```
TransactionID
isFraud
class_weight
split_name
processing_version
feature_schema_version
generated_at
```

C.3 Risk bands

Minimum	Maximum	Band	Decision
0	19	low	approve
20	39	guarded	approve
40	59	medium	review
60	79	high	review
80	100	critical	reject

The risk bands are a fixed demonstration policy. They are not derived from the saved V2 validation operating point.

CHAPTER D

REPRODUCIBLE RUNBOOK

D.1 Raw data placement

```
data/data/ieee-fraud-detection/
  train_transaction.csv
  train_identity.csv
  test_transaction.csv
  test_identity.csv
  sample_submission.csv # optional
```

D.2 Preprocessing

```
docker compose -f docker-compose.preprocessing.yml build
docker compose -f docker-compose.preprocessing.yml run --rm
   preprocess
docker compose -f docker-compose.preprocessing.yml run --rm verify-
   processed
```

If a completed Spark export lacks a finalized manifest:

```
python -m pipeline.processed_contract \
  --output-dir data/processed/ieee_cis_fraud_risk
python -m pipeline.verify_processed_data \
  --output-dir data/processed/ieee_cis_fraud_risk
```

D.3 Model training

```
bash scripts/run_official_full.sh
```

The official workflow runs preprocessing, contract validation, baseline, comparison, tuning, calibration, threshold analysis, holdout evaluation, packaging, checksum generation, and the promotion gate. The current official decision is `not_promoted`; the serving configuration remains unchanged. Holdout evaluation must occur only after model and policy selection are fixed.

D.4 Application

```
docker compose up --build
```

Then open:

- dashboard: `http://localhost:5173`;
- API documentation: `http://localhost:8000/docs`;
- optional Adminer: `http://localhost:8081`; select the `PostgreSQL` system, then use the configured database credentials.

D.5 Local development

```
# Terminal 1
cd backend
uv sync --frozen
uv run uvicorn fraud_backend.main:app --reload

# Terminal 2
cd frontend
npm ci
npm run dev
```

D.6 Rollback

```
FRAUD_MODEL_SERVING_VERSION=v1
```

Restart the backend and verify `/meta` before accepting traffic. The rollback setting is a configuration path, not evidence that V1 has been validated as the current champion in the official snapshot.

CHAPTER E

TEST AND CONTRIBUTION MATRIX

E.1 Recommended validation commands

```
# Processed data (read-only unless --write-report is supplied)
python -m pipeline.verify_processed_data \
  --output-dir data/processed/ieee_cis_fraud_risk

# Model
cd model
uv run ruff check .
uv run pytest -q

# Backend
cd backend
uv run ruff check src tests
uv run pytest -q

# Frontend
cd frontend
npm run lint
npm run build
python scripts/check-ui.py
```

E.2 Acceptance scenarios

Table E.1: *Acceptance scenarios*

Scenario	Expected evidence
Processed handoff	Verifier passes every schema, row, split, and artifact check
Real model startup	`/meta` returns expected version with no warning
Full-feature score	`scoring_mode=full_feature`; probability bounded
Partial score	`partial_demo` is visible; required fields enforced

Scenario	Expected evidence
Unseen category	Request succeeds with training-defined unknown mapping
SHAP absent	UI renders explicit unavailable state
Review update	Status/label persist and queue/statistics invalidate
CSV row error	Good rows commit; bad row appears in bounded error list
Job reconnect	Active job is returned after browser refresh
Model rollback	V1 loads and `/meta` reports V1

E.3 Team roster and repository role mapping

Table E.2: *Group 1 roster and recorded subsystem contributions*

Team member	Repository evidence
Dương Bình An	Data processing, EDA, feature engineering, contracts, and handover
Lê Quang Tuyến	Named team member; a subsystem-specific role is not stated in the reviewed repository
Phạm Đức Long	React dashboard, risk/SHAP presentation, review UX, and import UI
Đỗ Quốc Trung	FastAPI, SQL persistence, imports, background data loading, and Docker
Dương Hồng Quân	Model training, comparison, SHAP, artifact packaging, and scoring
Nguyễn Lê Hồng Nhi	Named team member; a subsystem-specific role is not stated in the reviewed repository

The full roster and supervisor were supplied by the project team. Detailed contribution statements should be confirmed by all members before submission.